

For more details, please visit <https://assistedlabs.com>

Helm Charts for Application Packaging in OpenShift

HELM CHARTS FOR APPLICATION PACKAGING IN OPENSIFT



Introduction

In modern Kubernetes-based environments, managing the deployment and configuration of applications can be complex due to the number of resources and configurations involved.

Helm is a package manager for Kubernetes that simplifies the process by allowing applications to be defined, installed, and upgraded as **Helm charts**.

OpenShift, being an enterprise-grade Kubernetes distribution, fully supports Helm, enabling developers to package applications, manage their lifecycle, and promote consistency across environments.

Key benefits of using Helm in OpenShift:

To get the OpenShift Hands-on Experience, kindly visit: <https://assistedlabs.com>

ASSISTED LABS

For more details, please visit <https://assistedlabs.com>

- Simplified application packaging and deployment.
 - Reusable templates for different environments.
 - Version control for application configurations.
 - Easy rollback to previous versions.
 - Integration with CI/CD pipelines.
-

Understanding Helm and Helm Charts

Helm operates similarly to a package manager (like yum or apt) but for Kubernetes applications.

A **Helm chart** is a collection of files that describe a set of Kubernetes resources.

Components of a Helm Chart:

1. **Chart.yaml** – Contains metadata about the chart (name, version, description).
2. **values.yaml** – Default configuration values for templates.
3. **templates/** – Contains Kubernetes manifest templates.
4. **charts/** – Optional directory for dependent charts.
5. **NOTES.txt** – Displayed after chart installation to provide usage instructions.

Example Chart.yaml:

```
apiVersion: v2
name: myapp
description: A Helm chart for deploying MyApp in OpenShift
type: application
version: 1.0.0
appVersion: 1.0
```

Example values.yaml:

```
replicaCount: 3
image:
  repository: quay.io/myorg/myapp
  tag: latest
  pullPolicy: IfNotPresent
service:
  type: ClusterIP
  port: 8080
```

To get the OpenShift Hands-on Experience, kindly visit: <https://assistedlabs.com>

For more details, please visit <https://assistedlabs.com>

Helm in the OpenShift Environment

Helm is not enabled in OpenShift by default but can be easily installed. OpenShift 4.x supports **Helm 3**, which does not require Tiller (server-side component from Helm 2), making it more secure and compatible with OpenShift's security model.

Steps to enable and use Helm in OpenShift:

1. **Install Helm CLI**
 2. `curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash`
 3. **Login to OpenShift**
 4. `oc login --token=<token> --server=<openshift-api-url>`
 5. **Add Helm repositories**
 6. `helm repo add stable https://charts.helm.sh/stable`
 7. `helm repo update`
 8. **Search for charts**
 9. `helm search repo nginx`
 10. **Install a chart**
 11. `helm install my-nginx stable/nginx-ingress`
 12. **List releases**
 13. `helm list`
-

Creating a Custom Helm Chart for OpenShift

OpenShift-specific applications often require custom configurations such as:

- SecurityContext adjustments for restricted SCC (Security Context Constraints).
- Routes instead of standard Kubernetes Ingress.
- ImageStreams for internal image management.

Example: Creating a Helm chart

```
helm create myapp
```

This creates the default Helm chart structure. Then:

To get the OpenShift Hands-on Experience, kindly visit: <https://assistedlabs.com>

ASSISTED LABS

For more details, please visit <https://assistedlabs.com>

- Modify deployment.yaml under templates/ to suit OpenShift.
- Use route.yaml instead of Ingress for external access.

Example route template (templates/route.yaml):

```
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: {{ .Chart.Name }}
spec:
  to:
    kind: Service
    name: {{ include "myapp.fullname" . }}
  port:
    targetPort: http
```

Deploy the chart:

```
helm install myapp ./myapp
```

Managing Helm Releases in OpenShift

Once deployed, Helm charts become **releases** in your OpenShift project. You can:

- **Upgrade:**
 - helm upgrade myapp ./myapp
- **Rollback:**
 - helm rollback myapp 1
- **Uninstall:**
 - helm uninstall myapp

Version Control:

Helm charts should be stored in a Git repository for tracking changes, enabling GitOps workflows with tools like ArgoCD.

Integrating Helm with OpenShift Pipelines

You can integrate Helm into CI/CD pipelines (Jenkins, Tekton, GitHub Actions) to automate:

To get the OpenShift Hands-on Experience, kindly visit: <https://assistedlabs.com>

ASSISTED LABS

For more details, please visit <https://assistedlabs.com>

- Chart linting and validation.
- Packaging and publishing charts.
- Deployments to multiple OpenShift environments.

Example Tekton task snippet:

steps:

```
- name: helm-upgrade
  image: alpine/helm:3.5.4
  script: |
    helm upgrade --install myapp ./charts/myapp \
    --values ./charts/myapp/values-prod.yaml
```

Best Practices for Helm in OpenShift

- Use **values.yaml** for environment-specific configurations.
 - Use **Route** instead of Ingress for OpenShift-native networking.
 - Ensure proper **securityContext** for restricted SCC.
 - Sign Helm charts for security.
 - Store charts in an internal Helm repository or Artifact Hub.
 - Use **helm lint** before deploying.
-

Conclusion

Helm provides a powerful and flexible way to package, deploy, and manage applications in OpenShift.

By leveraging Helm charts:

- Teams can ensure consistent deployments across environments.
- Configuration changes become easier to manage and track.
- Application lifecycle operations such as upgrades and rollbacks are streamlined.

In OpenShift, Helm bridges the gap between Kubernetes-native packaging and enterprise-grade operational requirements, making it a vital tool for DevOps teams.

To get the OpenShift Hands-on Experience, kindly visit: <https://assistedlabs.com>